Projeto 4: Reconhecimento de Comandos de Voz com LSTM Bidirecional

Disciplina: Introdução às Redes Neurais Artificiais

Autor: Gabriel Bonavina (RA: 168.858)

Repositório: https://github.com/gbonavina/projeto-4-rna

Configuração Experimental

Este trabalho classifica palavras faladas isoladas (*keyword spotting*) com uma rede recorrente. Cada gravação é convertida numa série temporal, o espectrograma log-mel, e uma LSTM lê essa série quadro a quadro para decidir qual palavra foi dita. Os dados são o Google Speech Commands v0.02 [1] (licença CC BY 4.0), com gravações de 1 segundo, mono, a 16 kHz, de 35 palavras em inglês: *backward*, *bed*, *bird*, *cat*, *dog*, *down*, *eight*, *five*, *follow*, *forward*, *four*, *go*, *happy*, *house*, *learn*, *left*, *marvin*, *nine*, *no*, *off*, *on*, *one*, *right*, *seven*, *sheila*, *six*, *stop*, *three*, *tree*, *two*, *up*, *visual*, *wow*, *yes* e *zero*. As gravações foram feitas por voluntários, cada um com o próprio microfone e ambiente, então a qualidade varia bastante de um clipe para outro. O conjunto traz ainda seis gravações de ruído de fundo, somando cerca de 6,7 minutos: lavar louça (95 s), um gato miando (62 s), bicicleta ergométrica (61 s), torneira aberta (61 s), ruído branco e ruído rosa (60 s cada, gerados sinteticamente).

Foi usada a divisão oficial do conjunto, que atribui cada locutor a uma partição pelo *hash* do seu identificador (Tabela 1). Assim nenhum locutor do teste aparece no treino, e a acurácia de teste mede a generalização para vozes novas. As 20 palavras "principais" têm cerca de 3 000 clipes de treino cada, e as 15 auxiliares têm entre 1 250 e 1 730; as menos representadas são *forward*, *follow*, *learn*, *visual*, *backward* e *tree* (Figura 1). O desbalanceamento é moderado (cerca de 2,5 vezes entre a maior e a menor classe) e não recebeu nenhum tratamento especial.

Tabela 1: Divisão oficial do Speech Commands v0.02, feita por locutor.

Partição	Clipes	Locutores
Treino	84 843	2 112
Validação	9 981	256
Teste	11 005	250

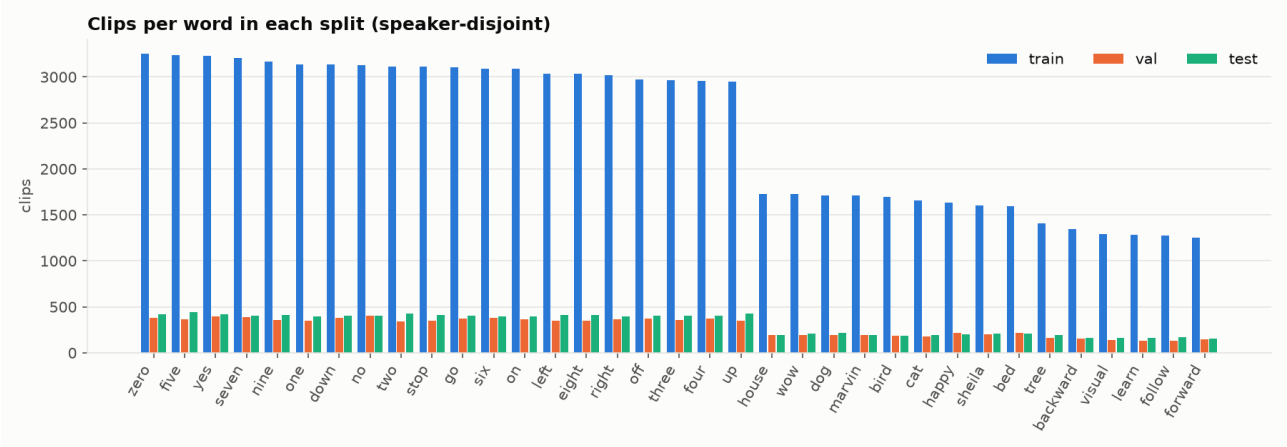


Figura 1: Número de clipes por palavra em cada partição. As 20 palavras da esquerda são as principais do conjunto; as 15 da direita têm cerca de metade dos exemplos.

Controle de vazamento

Com áudio de locutores reais, o vazamento mais comum é a mesma pessoa aparecer no treino e no teste: a rede aprende a voz e o microfone, e a acurácia de teste fica otimista. O carregamento dos

dados (`speech_data.check_no_leakage`) verifica que nenhum arquivo e nenhum locutor aparece em mais de uma partição. Além disso:

1. a média e o desvio padrão usados para normalizar cada banda do log-mel foram calculados só sobre os clipes de treino;
2. o aumento de dados é aplicado só aos lotes de treino, e validação e teste são avaliados com os clipes originais. As gravações de ruído não contêm fala, então não carregam rótulo nem identidade de locutor;
3. a melhor época foi escolhida pela acurácia de validação, e o teste foi avaliado uma única vez, no fim;
4. o script dos gráficos recarrega o *checkpoint* salvo e confere que ele reproduz a acurácia de teste registrada no treino.

O áudio como série temporal: espectrograma log-mel

O problema é tratado como classificação de uma série temporal multivariada. Cada clipe é transformado numa sequência de 101 vetores, um a cada 10 ms, e cada vetor descreve como a energia do som está distribuída entre 40 faixas de frequência naquele instante. A LSTM lê essa sequência passo a passo, do mesmo jeito que leria uma série de medições de um sensor, e só depois de percorrer os 101 passos decide qual palavra foi dita. Essa sequência é o espectrograma log-mel, uma representação padrão em reconhecimento de fala.

Usar a forma de onda diretamente como série temporal seria possível, mas pouco prático, por três motivos:

1. **Comprimento da sequência.** Um segundo a 16 kHz tem 16 000 amostras. Uma LSTM com 16 000 passos é lenta, porque os passos são processados em série, e tem dificuldade de propagar o gradiente por uma distância tão grande. Com o espectrograma a mesma informação cabe em 101 passos.
2. **O que identifica a palavra é o espectro, não a amostra.** Duas pessoas dizendo *yes* produzem formas de onda muito diferentes amostra a amostra, porque a fase e a altura da voz variam. O que se repete entre elas é a sequência de formantes (as faixas de frequência que o trato vocal reforça em cada vogal e consoante), que é exatamente o que o espectrograma mostra. Ao descartar a fase, o espectrograma remove uma fonte de variação que não ajuda a reconhecer a palavra.
3. **Volume.** A amplitude da gravação depende da distância ao microfone e do ganho de cada aparelho, que no Speech Commands variam muito porque os clipes foram gravados por voluntários com equipamentos próprios [1].

As duas operações que dão nome ao log-mel tratam dos dois últimos pontos. A escala mel agrupa as frequências do jeito que o ouvido humano as resolve: os filtros são estreitos nas frequências baixas, onde ficam os formantes que diferenciam as vogais, e largos nas altas, onde detalhes finos importam pouco para a fala. Isso reduz os 257 coeficientes da FFT a 40 bandas sem perder o que distingue as palavras; o uso de filtros na escala mel para reconhecimento de palavras vem de Davis e Mermelstein [2]. O logaritmo comprime a faixa dinâmica, de forma parecida com a percepção de volume, e transforma um ganho multiplicativo do microfone num deslocamento aditivo, que a normalização por banda remove em boa parte.

O cálculo (`src/speech_data.py`) segue os passos usuais. Clipes com menos de 16 000 amostras recebem zeros no final. O sinal é cortado em janelas de Hann de 25 ms (400 amostras) com passo de 10 ms (160 amostras), o que dá 101 quadros. Cada quadro passa por uma FFT de 512 pontos (257 coeficientes

entre 0 e 8 kHz), o espectro de potência é somado por 40 filtros triangulares na escala mel entre 20 Hz e 8 kHz, e aplica-se o logaritmo:

$$X_{t,m} = \log\Big(\sum_{k} H_m(k) \, \big|\mathrm{FFT}(w \cdot x_t)(k)\big|^2 + 10^{-6} \Big), \qquad t = 1, \dots, 101, \quad m = 1, \dots, 40, \tag{1}$$

em que x_t é o quadro t, w a janela de Hann e H_m o m-ésimo filtro mel. Cada banda m é então normalizada com a média e o desvio padrão do treino. A entrada da rede é a série $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_{101})$, com $\mathbf{x}_t \in \mathbb{R}^{40}$. O cálculo é feito na GPU a cada lote, e o banco de filtros mel foi implementado em numpy.

A Figura 2 mostra quatro exemplos. No espectrograma, o eixo horizontal é o tempo da série e cada coluna é um vetor $\mathbf{x}_t$. A palavra ocupa uma parte do segundo, e a posição varia: em *yes* ela começa perto de 0,65 s, em *three* logo no início. O par *tree*/*three* já mostra a dificuldade que aparece adiante. As duas palavras diferem quase só na consoante inicial, que dura poucos quadros.

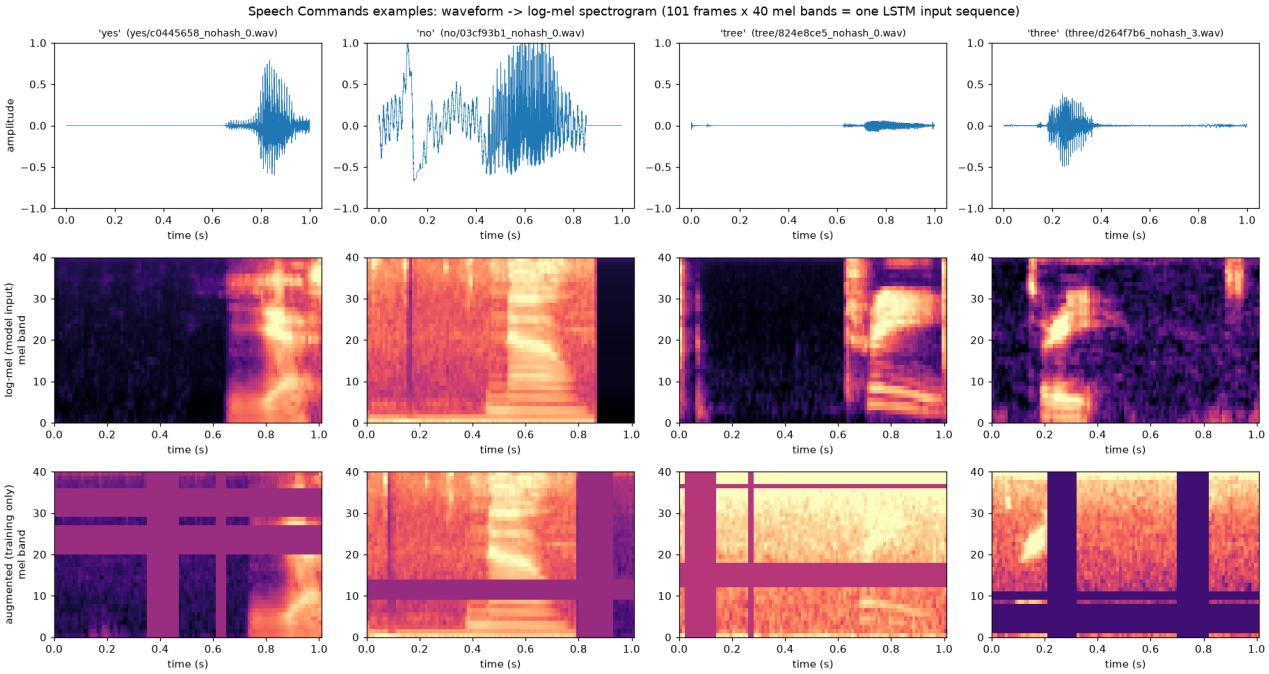


Figura 2: Quatro clipes do conjunto. Primeira linha: forma de onda. Segunda: espectrograma log-mel normalizado, que é a entrada da rede. Terceira: o mesmo espectrograma depois do aumento de dados usado no treino (deslocamento, ruído de fundo e máscaras do SpecAugment).

Modelo

O classificador (`src/model.py`) é uma LSTM [3] bidirecional de duas camadas (Figura 3). A LSTM é uma rede recorrente: a cada passo t ela combina o quadro $\mathbf{x}_t$ com o estado que carregou do passo anterior, $\mathbf{h}_t = \mathrm{LSTM}(\mathbf{x}_t, \mathbf{h}_{t-1})$, e as portas internas decidem o que guardar e o que esquecer. Assim o estado em cada instante resume o trecho da série já lido, por exemplo que uma consoante fricativa acabou de ser ouvida e uma vogal está começando. Primeiro uma *LayerNorm* normaliza os 40 valores de cada quadro. Em cada camada, uma LSTM lê a sequência do início para o fim e outra do fim para o início, cada uma com 256 unidades, e as saídas das duas são concatenadas em 512 valores por quadro. Entre as camadas há *dropout* de 0,2. Ler a sequência nos dois sentidos é válido aqui porque o clipe inteiro está disponível quando a classificação é feita; não existe "futuro" que a rede pudesse espiar.

Para resumir a sequência num único vetor, as saídas da segunda camada passam por média e máximo ao longo do tempo, que concatenados dão 1 024 valores. A alternativa comum, usar só a saída do último passo, deixaria a direção reversa com apenas um quadro de contexto, já que no último instante ela acabou de começar a leitura. Com a média e o máximo todos os quadros contribuem, e o máximo

deixa a rede responder a um trecho curto e característico em qualquer posição do clipe. A cabeça de classificação é *Dropout* → Linear(1024 → 512) → GELU → *Dropout* → Linear(512 → 35). O modelo tem 2,73 milhões de parâmetros, dos quais 2,19 milhões estão nas LSTMs.

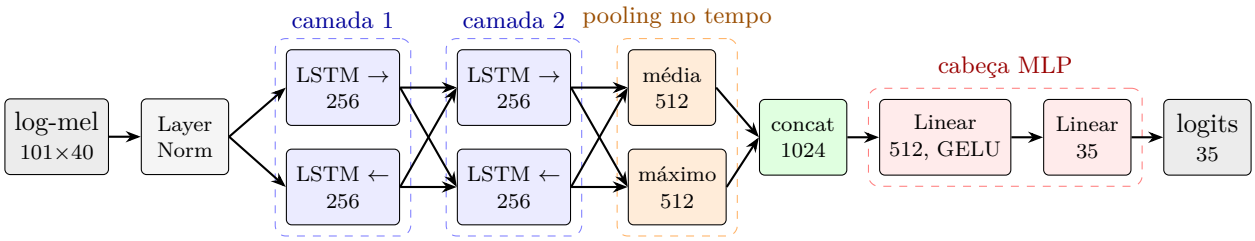


Figura 3: Arquitetura do classificador. Cada camada recorrente tem uma LSTM em cada sentido, com saídas concatenadas (512 valores por quadro); a segunda camada recebe a saída concatenada da primeira. Os números indicam a largura de cada bloco. Há *dropout* de 0,2 entre as camadas LSTM e antes de cada camada linear.

O modelo devolve *logits*, sem softmax na saída. A função de perda `CrossEntropyLoss` aplica o log-softmax internamente, que é numericamente mais estável do que calcular o softmax e depois o logaritmo. As probabilidades são obtidas com softmax(logits) na inferência. A softmax é a escolha certa porque as classes são mutuamente exclusivas: cada clipe tem uma única palavra, e as probabilidades devem somar 1. Sigmoides independentes serviriam para um problema com vários rótulos por exemplo, que não é o caso.

Treinamento

O otimizador foi o AdamW com decaimento de pesos 10^{-2} e taxa de aprendizado no esquema *one-cycle*, que sobe até 2×10^{-3} e depois desce até perto de zero ao longo das 40 épocas. Os lotes têm 256 clipes, a norma do gradiente é limitada a 1,0 e a semente é 0. A perda é a entropia cruzada com suavização de rótulos de 0,1: o alvo deixa de ser 1 para a classe certa e 0 para as outras e passa a ser

$$\tilde{y}_k = (1-\varepsilon)\mathbb{1}[k=y] + \frac{\varepsilon}{35}, \qquad \varepsilon = 0{,}1, \tag{2}$$

ou seja, cerca de 0,903 para a classe certa e 0,003 para cada uma das outras. Isso desestimula a rede de empurrar as probabilidades para os extremos. O conjunto de treino inteiro fica na GPU como formas de onda em `int16`, e os espectrogramas e o aumento de dados são calculados a cada lote, então cada época vê versões aumentadas diferentes dos clipes.

O aumento de dados, aplicado só ao treino, tem três etapas nesta ordem:

1. **Deslocamento no tempo:** o clipe é deslocado aleatoriamente em até ±100 ms, com zeros preenchendo o espaço que sobra. Isso ensina que a palavra pode aparecer em qualquer ponto do segundo.

2. **Ruído de fundo:** em 80% dos clipes soma-se um trecho aleatório de 1 segundo das gravações de ruído, com ganho sorteado uniformemente entre 0 e 0,1 da escala cheia.

3. **SpecAugment** [4]**:** sobre o log-mel já normalizado, duas faixas de frequência (até 8 bandas cada) e duas faixas de tempo (até 12 quadros cada) são zeradas. Como o espectrograma está normalizado, zero corresponde à média do treino. A rede precisa então reconhecer a palavra mesmo com partes do espectro faltando.

A terceira linha da Figura 2 mostra o efeito combinado. Em *tree*, o ruído somado é mais forte que a própria fala e quase apaga a palavra.

1. Resultados de Treinamento

A Figura 4 mostra as curvas e a Tabela 2 alguns pontos delas. A acurácia de validação passa de 90% na época 10 e cresce devagar depois disso, chegando ao máximo de 95,96% na época 36, cujos pesos foram mantidos. Nas últimas quatro épocas ela fica estável em torno de 95,9%, sem queda que indique sobreajuste.

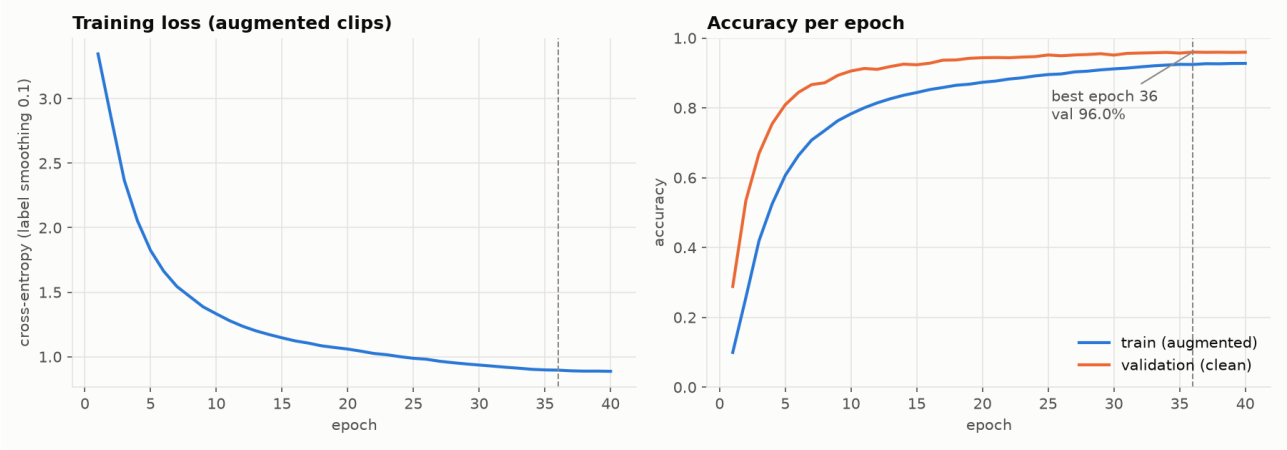


Figura 4: Perda de treino (com suavização de rótulos, sobre clipes aumentados) e acurácia de treino e validação por época. A linha tracejada marca a época 36, escolhida pela validação.

Tabela 2: Perda e acurácia em algumas épocas. A perda de treino é medida nos clipes aumentados e inclui a suavização de rótulos.

Época	Perda de treino	Acurácia de treino	Acurácia de validação
1	3,347	9,9%	28,8%
10	1,332	78,3%	90,6%
20	1,059	87,3%	94,4%
30	0,936	91,2%	95,1%
36 (melhor)	0,896	92,4%	96,0%
40	0,887	92,7%	95,9%

A acurácia de validação fica acima da de treino durante todo o treino, o contrário do que se costuma ver. A diferença vem do aumento de dados: os clipes de treino recebem ruído, deslocamento e máscaras, e por isso são mais difíceis que os de validação, que são limpos. O *dropout*, ativo só no treino, contribui no mesmo sentido. A perda de treino não chega perto de zero porque a suavização de rótulos impõe um piso. Mesmo um modelo que acertasse o alvo $\tilde{y}$ perfeitamente teria entropia cruzada de cerca de 0,66.

2. Desempenho no Teste

Avaliado uma única vez sobre os 11 005 clipes de teste, de 250 locutores que a rede nunca ouviu, o modelo acertou 10 465, uma acurácia de 95,09%. Um palpite aleatório acertaria $1/35 = 2{,}86\%$. A diferença de menos de um ponto em relação à validação (95,96%) é compatível com a troca de locutores entre as duas partições. A Tabela 3 resume as métricas.

Tabela 3: Métricas finais do modelo da época 36.

Métrica	Valor
Acurácia de teste	95,09% (10 465 / 11 005)
Acurácia de validação (época 36)	95,96%
Acurácia de um palpite aleatório	2,86%
F1 macro (teste)	0,948
Erro de calibração esperado (ECE, 15 faixas)	0,085

O F1 macro, média simples do F1 das 35 classes, é 0,948, muito perto da acurácia. As palavras auxiliares, com metade dos exemplos de treino, não ficam sistematicamente para trás. A Figura 5 mostra precisão, revocação e F1 por palavra. As mais fáceis são *stop* (0,982), *yes* (0,976), *seven* e *zero* (0,973) e *marvin* (0,969). As mais difíceis são *tree* (0,862), *learn* (0,894), *forward* (0,902), *up* (0,923), *follow* (0,925) e *bed* (0,931). Das seis piores, quatro estão entre as menos representadas no treino, mas o que elas têm em comum, como mostra a seção seguinte, é ter uma palavra parecida na lista.

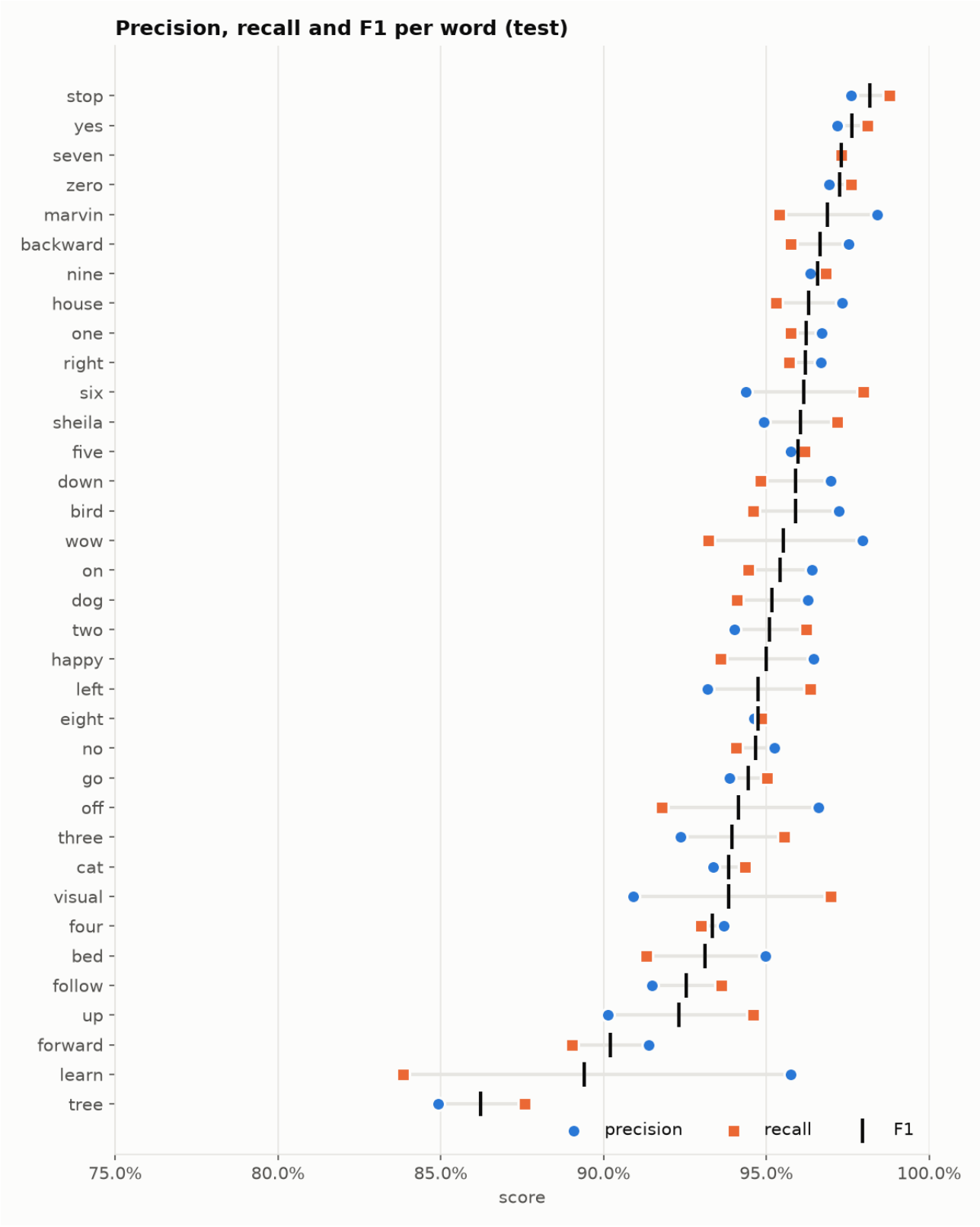


Figura 5: Precisão, revocação e F1 de cada palavra no conjunto de teste.

learn é o caso em que precisão e revocação mais se afastam: precisão de 0,957 e revocação de 0,839. Quando o modelo diz *learn* ele quase sempre acerta, mas 16% dos *learn* verdadeiros recebem outro rótulo, e o destino mais comum é *left*. *tree* é ruim nos dois sentidos (precisão 0,849, revocação 0,876), porque erra para *three* e também recebe exemplos de *three*.

3. Análise dos Erros

A Figura 6 traz a matriz de confusão normalizada por linha, e a Figura 7 os 15 erros mais frequentes. A diagonal concentra quase toda a massa, e os poucos valores fora dela que chegam a 2% aparecem em grupos pequenos de palavras. Os erros mais comuns (palavra verdadeira → prevista) são *off*→*up* (17 clipes), *tree*→*three* (15), *learn*→*left* (11), *four*→*forward* (11), *forward*→*four* (10), *three*→*tree* (10) e *four*→*follow* (8), seguidos de *two*→*zero*, *up*→*off*, *up*→*on* e *on*→*up*, com 6 cada.

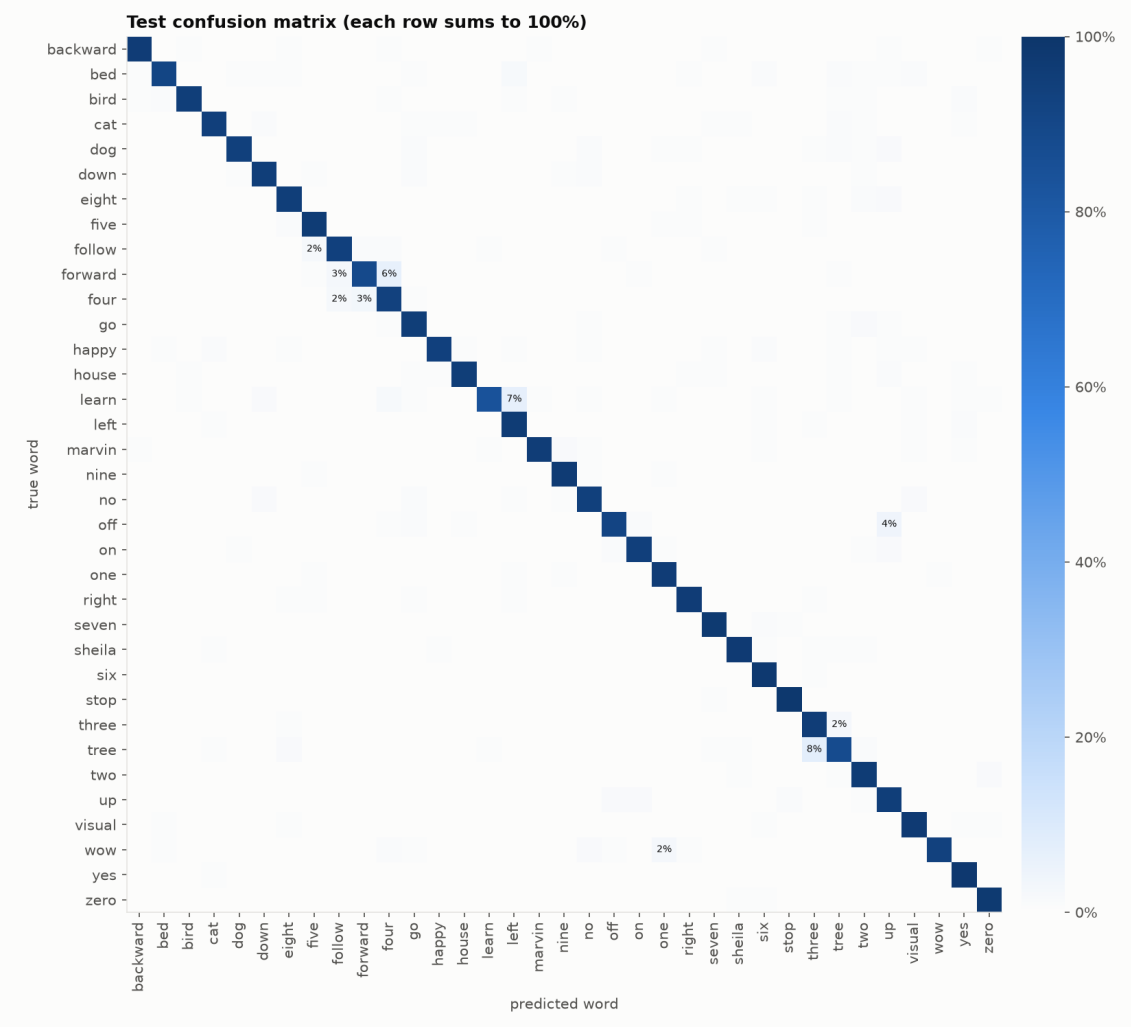


Figura 6: Matriz de confusão do teste, normalizada por linha (cada linha soma 100%). Os valores fora da diagonal a partir de 2% estão anotados.

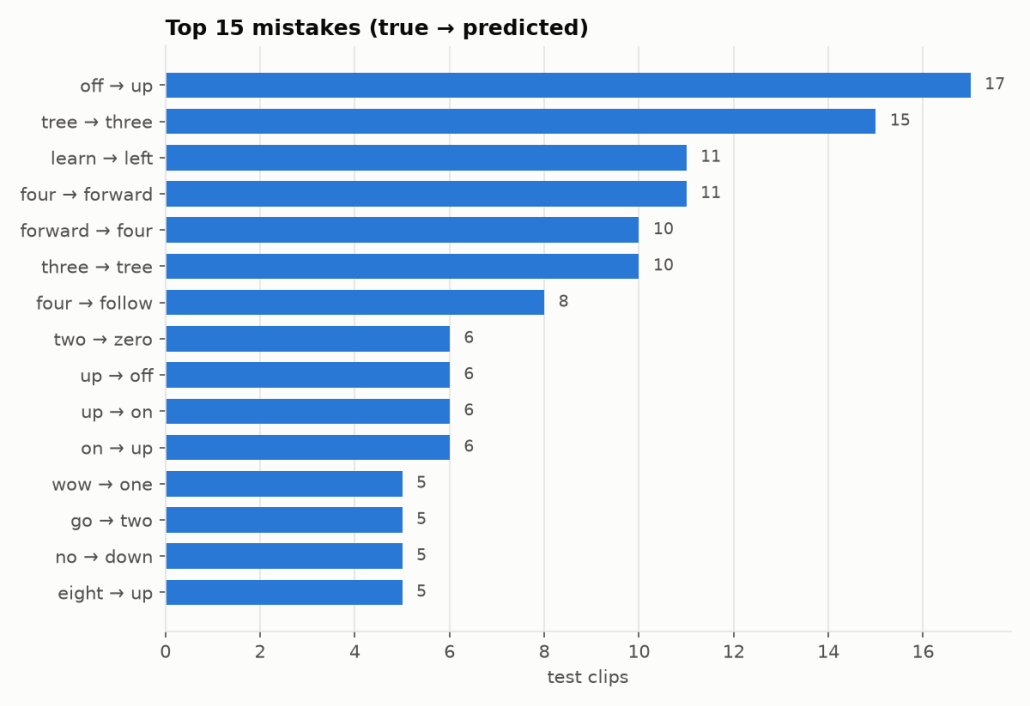


Figura 7: Os 15 pares (palavra verdadeira → palavra prevista) mais frequentes entre os erros do teste.

Quase todos esses pares são acusticamente próximos, e a semelhança ajuda a explicar cada um. *tree* e *three* diferem só no começo, *tr* contra *thr*, e o *th* é um som fraco, de pouca energia, que some com facilidade num microfone ruim ou com ruído de fundo; 8% dos *tree* viram *three*. *four*, *forward* e *follow* começam todos com /f/ seguido de vogal posterior, e *forward* é praticamente *four* com uma segunda sílaba fraca no fim. *learn* e *left* começam igual, com /l/ e vogal média. *off*, *up* e *on* são monossílabos curtos, com vogal parecida e consoante final de pouca energia. Os erros, portanto, não se espalham ao acaso entre as 35 classes: eles se concentram em pares mínimos e quase pares mínimos, onde a informação que distingue as palavras está em poucos quadros do espectrograma. Essa é também a parte do sinal que o SpecAugment às vezes apaga no treino e que o ruído encobre primeiro.

3.1. Erros com alta confiança e ruído nos rótulos

A Figura 8 mostra os 12 erros em que o modelo estava mais confiante. Alguns são confusões plausíveis, como *three*→*tree* com 94% e 92% de confiança, e *learn*→*left* com 93%. Outros são difíceis de explicar pelo som e parecem ser problemas do próprio rótulo. No clipe de *sheila* classificado como *two* (97%), o espectrograma mostra uma faixa de ruído constante e só uma mancha curta de energia nas bandas baixas, sem a estrutura de duas sílabas que *sheila* teria. No clipe de *six* classificado como *sheila* (92%), a gravação é cortada pouco depois de 0,55 s, e o que sobra é um som sibilante longo, que de fato lembra o início de *sheila*. O Speech Commands foi coletado com voluntários pela internet, e esse tipo de ruído de rótulo é uma característica conhecida do conjunto. A inspeção aqui é visual e não confirma qual era a palavra gravada, mas sugere que parte dos 4,9% de erro não está ao alcance de nenhum classificador.

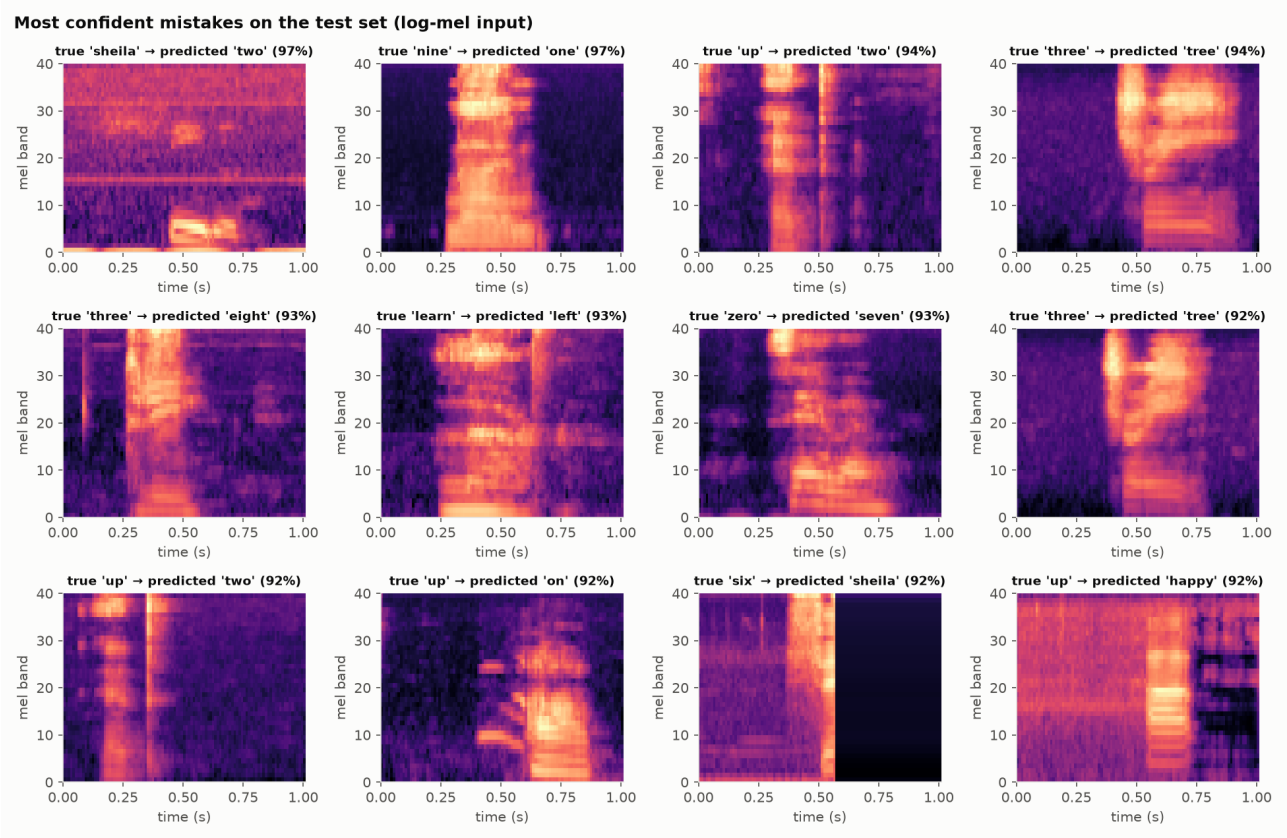


Figura 8: Espectrogramas log-mel dos 12 erros de teste com maior confiança, com a palavra verdadeira, a prevista e a probabilidade atribuída à classe prevista.

4. Robustez ao Ruído

Para medir o quanto o modelo tolera ruído, os clipes de teste foram reavaliados com ruído de fundo somado a uma relação sinal-ruído (SNR) fixa:

$$\mathrm{SNR} = 10 \log_{10} \frac{P_{\text{fala}}}{P_{\text{ruído}}} \ \text{dB}, \tag{3}$$

em que P_{fala} é a potência média do trecho de 0,25 s mais forte do clipe, para que o silêncio antes e depois da palavra não reduza a estimativa. Em 0 dB o ruído tem a mesma potência da fala. A Tabela 4 e a Figura 9 trazem os resultados.

Tabela 4: Acurácia de teste com ruído de fundo somado a uma SNR fixa.

SNR	sem ruído	20 dB	10 dB	5 dB	0 dB	−5 dB
Acurácia	95,1%	93,7%	87,0%	77,6%	61,1%	36,5%

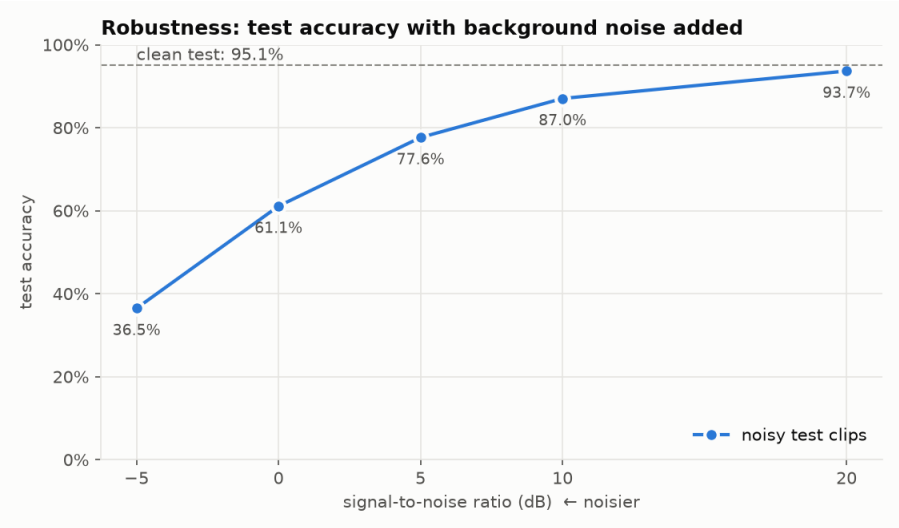


Figura 9: Acurácia de teste em função da SNR do ruído somado aos clipes.

A 20 dB a perda é de 1,4 ponto. A partir de 10 dB a queda acelera, e a 0 dB o modelo ainda acerta 61%, mais de vinte vezes o acaso. Há uma ressalva importante: o ruído do teste vem das mesmas seis gravações usadas no aumento de dados do treino, ainda que em trechos sorteados de novo. Esses números medem a robustez a tipos de ruído que a rede já conhece, e um ruído de natureza diferente (vozes ao fundo, música, reverberação) provavelmente causaria uma queda maior. Um teste mais justo exigiria gravações de ruído separadas para o treino e para a avaliação.

5. Calibração

Uma rede bem calibrada acerta cerca de 80% das vezes em que diz ter 80% de confiança. Para medir isso, os clipes de teste foram divididos em 15 faixas de confiança (a maior probabilidade da softmax), e o erro de calibração esperado (ECE) é a média ponderada da diferença entre acurácia e confiança em cada faixa:

$$\mathrm{ECE} = \sum_{b=1}^{15} \frac{n_b}{N} \left|\mathrm{acc}(b) - \mathrm{conf}(b)\right|. \tag{4}$$

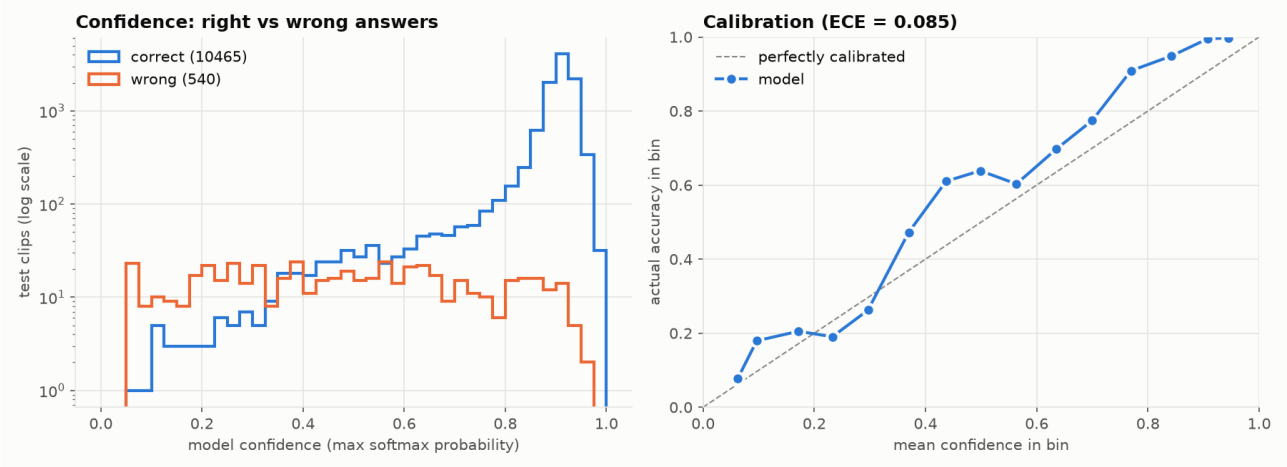


Figura 10: Esquerda: histograma da confiança nos acertos e nos erros do teste (eixo vertical em escala logarítmica). Direita: diagrama de confiabilidade; a diagonal tracejada corresponde a uma calibração perfeita.

O ECE é 0,085, e a Figura 10 mostra a direção do desvio. A partir de uma confiança de cerca de 0,35, os pontos do diagrama ficam acima da diagonal: na faixa em que o modelo diz ter 0,84 de confiança,

por exemplo, ele acerta 95%. O modelo é subconfiante, efeito esperado da suavização de rótulos, que treina a rede para nunca passar de cerca de 0,90 na classe certa. O histograma confirma isso. Quase todos os acertos têm confiança entre 0,85 e 0,95, e muito poucos passam de 0,97. Os 540 erros se espalham de forma quase uniforme entre 0,05 e 0,95. A confiança separa bem os dois grupos, então ela serve para filtrar respostas duvidosas, mas o valor numérico não deve ser lido como uma probabilidade exata. Se isso fosse necessário, um ajuste de temperatura sobre a validação corrigiria a escala sem alterar as previsões.

6. O que a Rede Aprendeu

Para ver como a rede organiza as palavras, foi extraída a ativação de 512 dimensões da camada oculta da cabeça (depois da GELU) para 80 clipes de teste de cada palavra, e esses vetores foram projetados em 2D com o t-SNE (perplexidade 30, inicialização pela PCA). O t-SNE procura uma disposição no plano em que pontos vizinhos no espaço original continuem vizinhos.

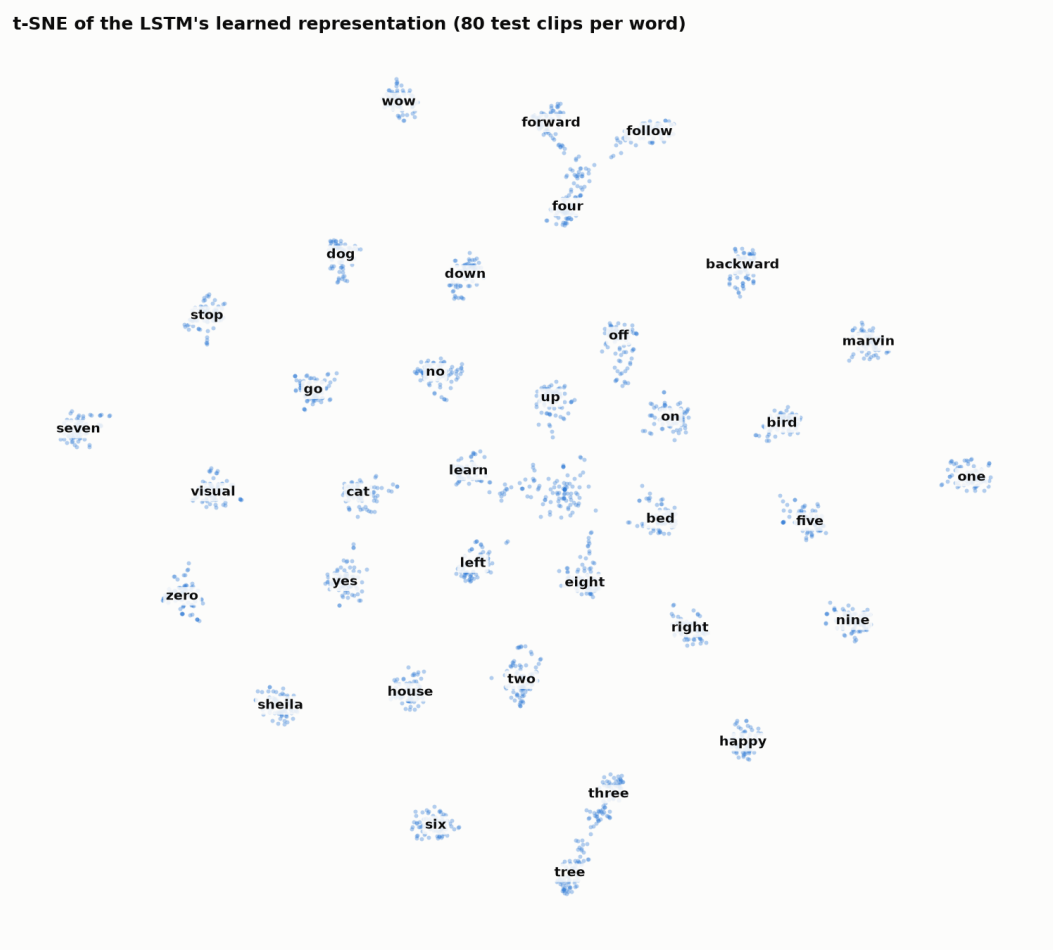


Figura 11: Projeção t-SNE da camada oculta de 512 dimensões para 80 clipes de teste de cada palavra. O rótulo de cada grupo está na posição média dos seus pontos.

Na Figura 11 cada palavra forma um grupo próprio e compacto. As palavras que se tocam são as mesmas que a matriz de confusão aponta: *three* e *tree* estão ligadas por uma ponte de pontos na parte de baixo, *four*, *forward* e *follow* formam um grupo de três braços no alto, e *off*, *up* e *on* ficam lado a lado no centro. Perto do centro há também uma nuvem difusa de pontos que não pertencem com clareza a nenhum grupo e que fica próxima de *learn* e *left*; são provavelmente clipes difíceis ou mal rotulados. As palavras sem vizinho parecido, como *seven*, *one*, *wow* e *sheila*, ficam isoladas na borda. Como em qualquer t-SNE, só a vizinhança tem significado; o tamanho dos grupos e a distância entre grupos afastados não podem ser lidos como medidas.

7. Conclusão

Tratando cada clipe como uma série temporal de 101 espectros log-mel, uma LSTM bidirecional de duas camadas classificou as 35 palavras do Speech Commands v0.02 com 95,1% de acurácia em locutores que não estavam no treino, contra 2,9% do acaso. Os resultados vêm de uma divisão por locutor verificada, de estatísticas de normalização e aumento de dados restritos ao treino e de um teste avaliado uma só vez. O aumento de dados explica a validação acima do treino e deu ao modelo tolerância a ruído conhecido: a acurácia fica acima de 87% até 10 dB de SNR. Os erros que restam são pares de palavras quase iguais, como *tree*/*three*, *four*/*forward*, *off*/*up* e *learn*/*left*, e uma parte dos erros mais confiantes parece vir de rótulos errados no próprio conjunto. A suavização de rótulos deixou o modelo subconfiante (ECE de 0,085). Faltaria testar a robustez com ruídos que a rede não viu no treino.

Referências

[1] P. Warden. *Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition*. arXiv:1804.03209, 2018. https://arxiv.org/abs/1804.03209

[2] S. Davis e P. Mermelstein. Comparison of parametric representations for monosyllabic word recognition in continuously spoken sentences. *IEEE Transactions on Acoustics, Speech, and Signal Processing*, 28(4):357–366, 1980.

[3] S. Hochreiter e J. Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[4] D. S. Park, W. Chan, Y. Zhang, C.-C. Chiu, B. Zoph, E. D. Cubuk e Q. V. Le. SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition. *Interspeech*, 2019. arXiv:1904.08779.